

TREINO

Como foi feito o treino — PepMem-AI

Documentação do pipeline de treinamento dos modelos que alimentam o dashboard (baseline e multimodal).

Para interpretar **saídas** (PMI, probabilidade, SHAP), veja `INTERPRETACAO_RESULTADOS.md`.

1. Visão geral

O treino responde à pergunta:

Este par **peptídeo × membrana** tem chance de **alta atividade** ($MIC \leq 3,4 \mu M$)?

São treinados **dois** Random Forests:

Modelo	Features	Uso típico
Baseline	11 descritores clássicos + PMI	Streamlit Cloud (leve, sem PyTorch)
Multimodal	clássicas + embedding ESM-2 (~320 dims)	Local / Hugging Face Space

Ambos passam por:

1. Montagem dos pares com MIC
 2. Validação **LOO por amostra** (referência) e **LOPO** (principal)
 3. **Calibração isotônica** nas probs OOF do LOPO
 4. Treino final em **todos** os dados
 5. (Opcional) relatório SHAP global
-

2. De onde vêm os dados

```
data/raw/ (OPM, APD...)
  ↓ build_datasets.py
data/processed/pepmem_base*.parquet + endpoints
  ↓ build_pairs.py (+ PMI)
data/processed/pepmem_pairs.parquet
  ↓ só linhas com mic_value
conjunto de treino MIC
```

Fonte	Papel
Literatura / projeto	MICs publicados (ex.: Parente, Amorim-Carmo)
data/bench/mic_bench.csv	MICs (e hemólise) da bancada — sobrescrevem literatura no mesmo par
generate_embeddings.py	ESM-2 facebook/esm2_t6_8M_UR50D → embeddings/esm2_all.npz

Atualizar bancada e retreinar:

```
python scripts/import_bench_mic.py --retrain
```

Pipeline completo (do zero, após downloads):

```
python scripts/run_pipeline.py
```

3. Rótulo binário

Para cada par com mic_value:

```
label_high_activity = 1 se MIC ≤ 3,4 μM
label_high_activity = 0 caso contrário
```

O limiar 3,4 μM é a regra operacional atual do projeto (ajustável no código em `train_utils.load_mic_pairs`).

Amostras recentes (ordem de grandeza): ~**90** pares MIC, ~**10** peptídeos distintos, ~**40** positivos.

4. Features

4.1 Baseline (CLASSIC_FEATURES)

Feature	Origem
q_peptide	Carga líquida
h_peptide	Hidrofobicidade média
mu_h_peptide	Momento hidrofóbico (Eisenberg)
surface_charge	Carga superficial da membrana
anionic_fraction	Fração aniônica
cholesterol	Colesterol
lps	LPS (Gram−)
peptidoglycan	Peptidoglicano (Gram+)
ergosterol	Ergosterol (fungo)
viral_envelope	Envelope viral
pmi	Índice PMI (ver interpretação)

Valores ausentes → preenchidos com 0 no treino baseline.

4.2 Multimodal

`X = [ features clássicas | embedding ESM-2 mean-pooled ]`

- Modelo de linguagem: **ESM-2 t6 8M** (leve).
- Dimensão típica: $11 + 320 \approx \mathbf{331}$ features.
- Só entram pares cujo `peptide_id` tem embedding em `esm2_all.npz`.

5. Algoritmo: Random Forest

Pipeline scikit-learn:

`StandardScaler → RandomForestClassifier`

Hiperparâmetro	Baseline	Multimodal
n_estimators	200	300
max_depth	livre (None)	6

Hiperparâmetro	Baseline	Multimodal
class_weight	balanced	balanced
random_state	42	42

class_weight='balanced' compensa o desbalanceamento entre ativos e inativos.

Código compartilhado: scripts/train_utils.py → make_rf_pipeline.

6. Validação cruzada

6.1 LOO por amostra (referência)

- Cada **linha** (par peptídeo×alvo) fica de fora uma vez (LeaveOneOut).
- Útil como teto otimista.
- **Problema:** o mesmo peptídeo pode estar no treino (outro alvo) enquanto um alvo dele está no teste → vazamento por homologia (ex.: StigA6 / StigA16 ~94% idênticos).

6.2 LOPO — leave-one-peptide-out (principal)

- Grupo = peptide_id (LeaveOneGroupOut).
- Em cada fold, **todos** os pares daquele peptídeo saem do treino.
- Mede generalização para um peptídeo **nunca visto**.
- As probabilidade OOF (out-of-fold) desse LOPO alimentam a calibração.

Para cada peptídeo P:

treinar RF em todos os pares cujo peptide_id ≠ P

predizer todos os pares de P

Agregar probs OOF → AUC global + AUC por peptídeo (quando há as duas classes)

6.3 Métricas típicas (último treino commitado)

Ordem de grandeza (podem mudar após --retrain):

Modelo	LOO amostra AUC	Leave-peptide AUC
Baseline	~0,88	~0,88

Modelo	LOO amostra AUC	Leave-peptide AUC
Multimodal	~0,85	~0,81

O multimodal costuma cair mais no LOPO: embeddings capturam similaridade de sequência e, sem o peptídeo no treino, generalizam um pouco pior nessa família pequena.

Arquivos: data/processed/models/baseline_mic_loo.json,
multimodal_mic_loo.json.

7. Calibração isotônica

Após o LOPO:

1. Tem-se, para cada amostra, `prob_raw_lope` (OOF).
2. Ajusta-se `IsotonicRegression` mapeando `prob_raw_lope` → rótulo `y` (com clip em `[0, 1]`).
3. Na inferência: `p_calibrada = calibrador.predict([p_bruta])`.

Isso alinha as porcentagens mostradas no dashboard às frequências observadas no LOPO (melhor do que confiar só no `predict_proba` cru do RF).

Artefatos:

- `baseline_mic_calibrator.joblib`
- `multimodal_mic_calibrator.joblib`
- `*_oof_probs.csv` (auditoria: raw vs calibrada por par)

8. Modelo final e artefatos

Depois da validação, o RF é **retreinado em 100% dos pares MIC** (já não é OOF) e salvo:

Arquivo	Conteúdo
<code>baseline_mic_rf.joblib</code> / <code>multimodal_mic_rf.joblib</code>	Pipeline scaler + RF

Arquivo	Conteúdo
*_calibrator.joblib	IsotonicRegression
*_mic_loo.json	Métricas e metadados
*_oof_probs.csv	Probs OOF LOPO
project_ranking_baseline.csv	Ranking offline (baseline)
shap_global_*.json + shap_beeswarm_*.png	Explicabilidade (compute_shap.py)

Pasta: data/processed/models/.

9. Scripts envolvidos

Script	Função
build_datasets.py	Consolida bases e endpoints (incl. bancada)
build_pairs.py	Pares + PMI + pivot MIC/hemólise
generate_embeddings.py	ESM-2
train_utils.py	LOO, LOPO, RF, calibração
train_baseline.py	Treino baseline + ranking
train_multimodal.py	Treino multimodal
compute_shap.py	SHAP global / beeswarm
import_bench_mic.py	Importa bancada e dispara retreino
run_pipeline.py	Orquestra ponta a ponta

10. Fluxo mental (resumo)

MICs (literatura + bancada)

↓

rótulo: MIC $\leq$ 3,4 μ M?

↓

features clássicas (+ ESM-2 no multimodal)

↓

LOO amostra → AUC “otimista”

LOPO → AUC “honestas” + probs OOF

↓

calibração isotônica (OOF LOP0)

↓

RF final em todos os dados + calibrador

↓

dashboard / API / ranking / SHAP

11. Limitações do treino atual

- Poucos peptídeos distintos (~10 com MIC) → LOPO é essencial, mas a amostra ainda é pequena.
 - Família Stigmurin muito parecida → risco de superestimar se usar só LOO por amostra.
 - Limiar 3,4 μM é uma escolha de projeto, não uma lei biológica.
 - Hemólise entra nos pares (HEMOLYSIS), mas o RF atual classifica **atividade MIC**, não toxicidade direta (toxicidade no ranking é proxy via cell_normal).
 - Cloud Streamlit usa **baseline** (sem torch); multimodal completo exige ambiente com PyTorch (local / HF Space).
-

12. Como reproduzir

Após dados processados e (para multimodal) embeddings:

```
python scripts/train_baseline.py
python scripts/train_multimodal.py
python scripts/compute_shap.py
```

Ou, com novos MICs da bancada:

```
python scripts/import_bench_mic.py --retrain
```

Conferir métricas:

```
python -c "import json;
print(json.load(open('data/processed/models/baseline_mic_loo.json'))
['leave_one_peptide_auc'])"
```